Aim: Hands-on Solidity programming assignments for creating smart contracts.

Tasks Performed:

- Opened the LearnETH Solidity Basics course in Remix IDE and studied the structure of a Solidity program using the Counter contract.
- Completed assignments on primitive data types, basic syntax, variables and global variables.
- Wrote functions that read and write state variables, and practiced pure and view functions.
- Used modifiers and constructors, function inputs and outputs, and visibility (private, internal, public).
- Applied control flow with if/else, ternary operators and for/while loops.
- Worked with arrays, mappings, structs and enums as data structures.
- Studied data locations (storage and memory), Ether and wei units, gas and gas price.
- Built a Charity contract that receives Ether and lets the owner withdraw it, then checked each assignment in Remix ("Well done! No errors.").

Theory:

Solidity program structure – Solidity is a language for writing smart contracts on blockchain platforms. A program commonly contains a license identifier, a compiler version declaration (pragma), a contract definition, state variables and functions.

Course topics – The LearnETH tutorials cover primitive data types, variables, functions, visibility, modifiers, constructors, control flow, arrays, mappings, structs, enums, data locations and transaction-related concepts.

pure and view – A function is pure if it neither reads nor modifies blockchain state, and view if it reads state without modifying it. Declaring them saves gas and makes code easier to maintain.

Data locations – Storage is the permanent memory of the contract on the blockchain, while memory is temporary and exists only during a function call. Choosing the right location keeps gas cost low.

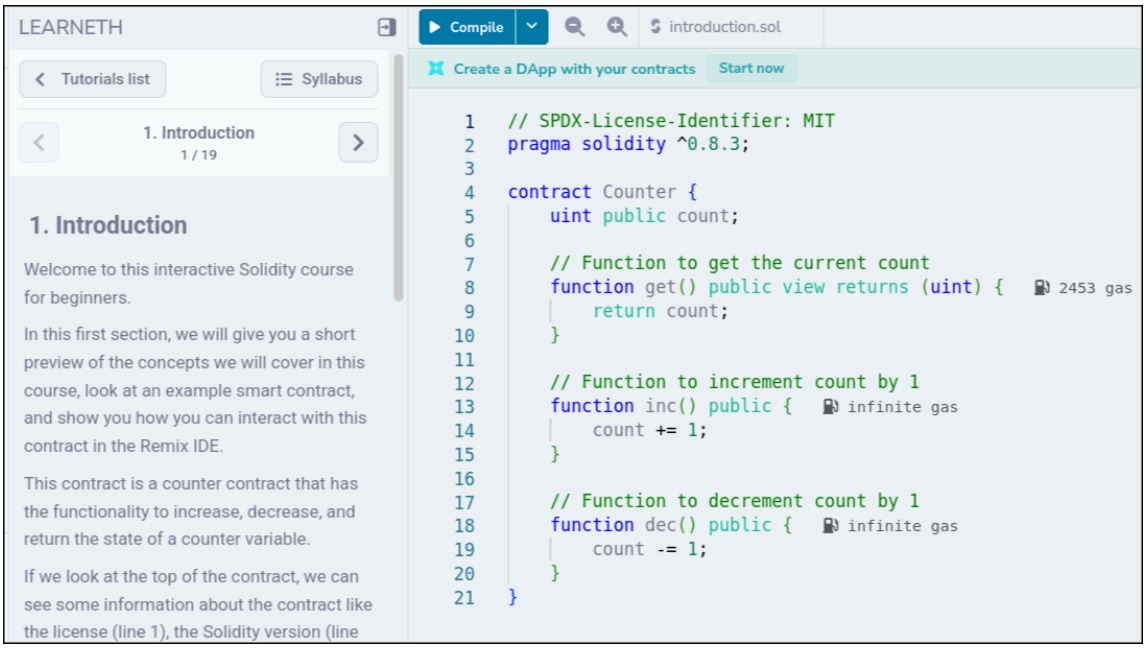
Ether, gas and gas price – Ether values use units such as wei, gwei and ether (1 ether = 10^18 wei). Gas measures the computation needed to run a transaction, and the gas price determines how much the sender pays for it.

Remix IDE – A browser-based environment used to compile contracts and deploy them on the Remix VM for testing.

Code & Output:

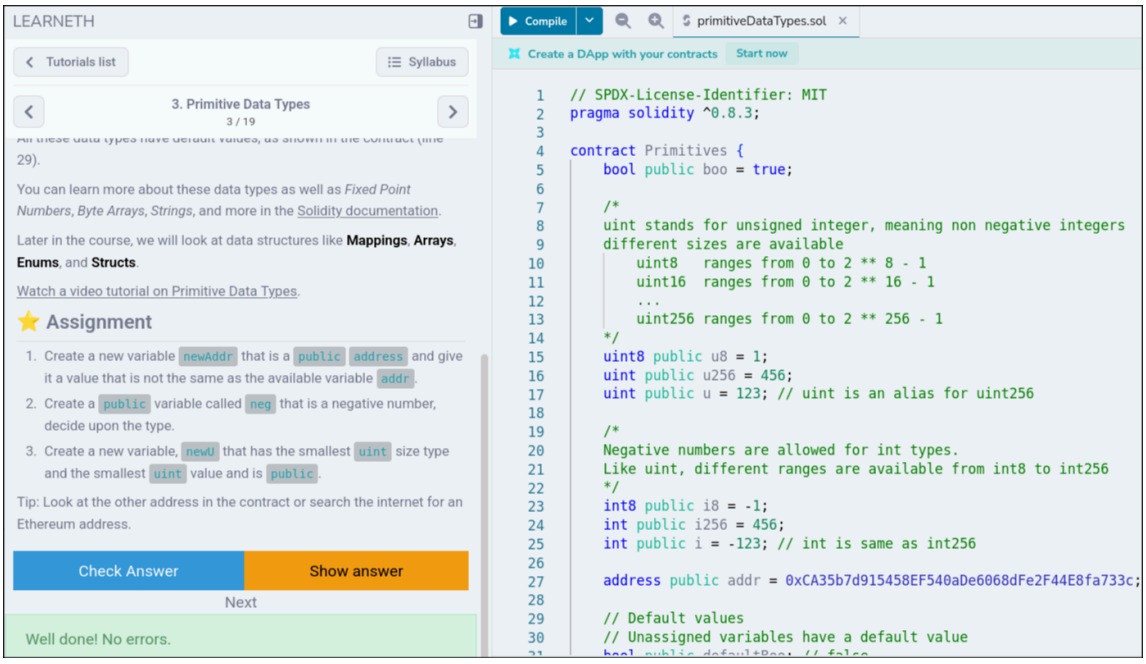
Step 1 — Introduction: a basic Counter contract

The Counter contract shows the license identifier, the pragma version, a state variable count, and the functions get (view), inc and dec.



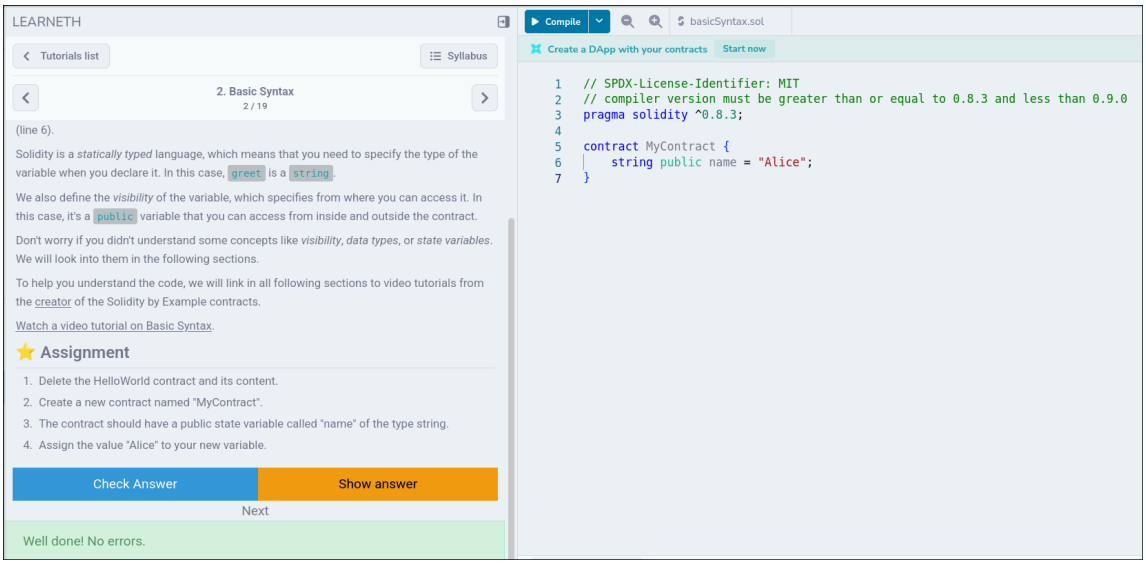
Step 2 — Basic syntax

A new contract MyContract is created with a public string state variable name assigned the value “Alice”.



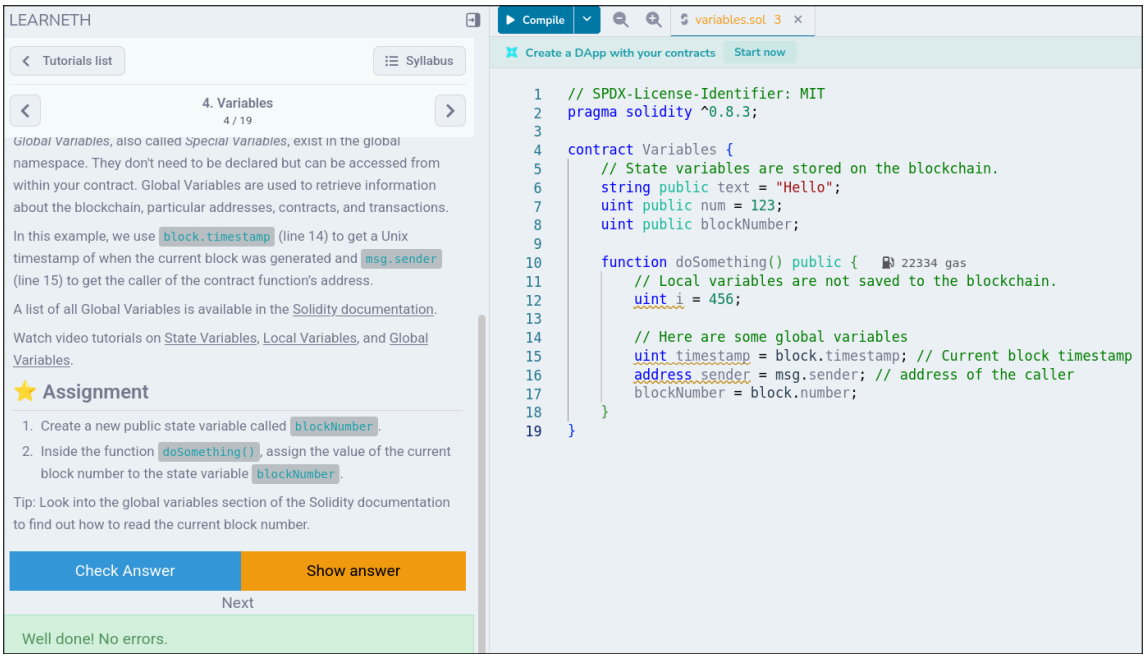
Step 3 — Primitive data types

Variables of types address, int and uint are created, including a public address newAddr, a public negative number neg, and the smallest uint newU.



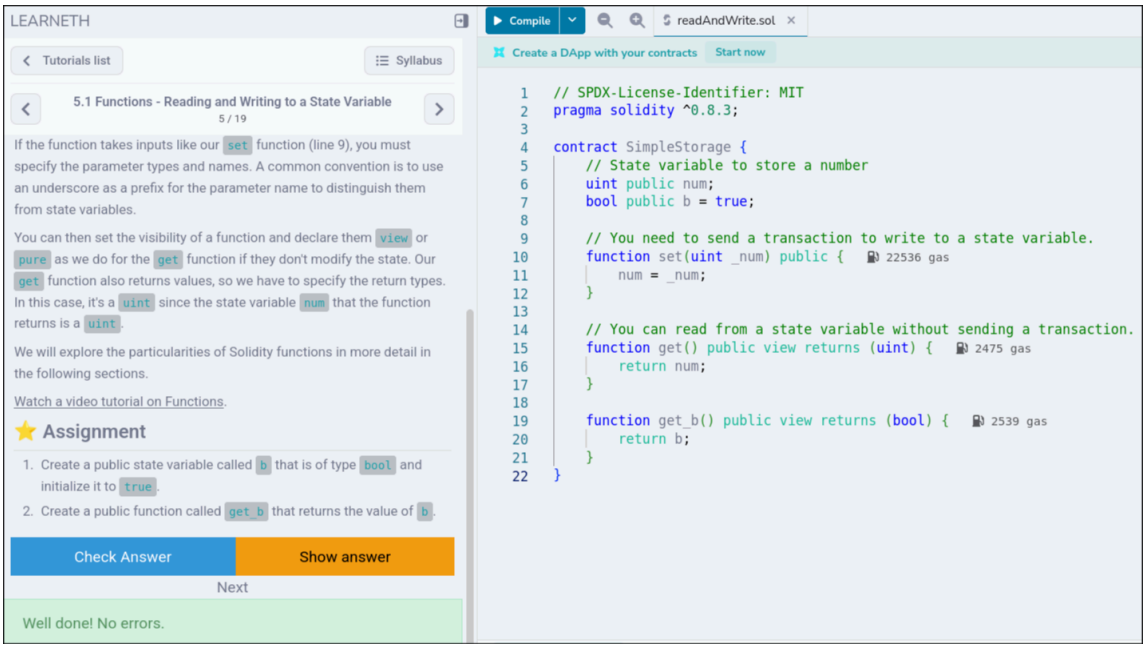
Step 4 — Variables and global variables

A public state variable blockNumber is created, and inside doSomething() it is assigned block.number. The example also reads block.timestamp and msg.sender.



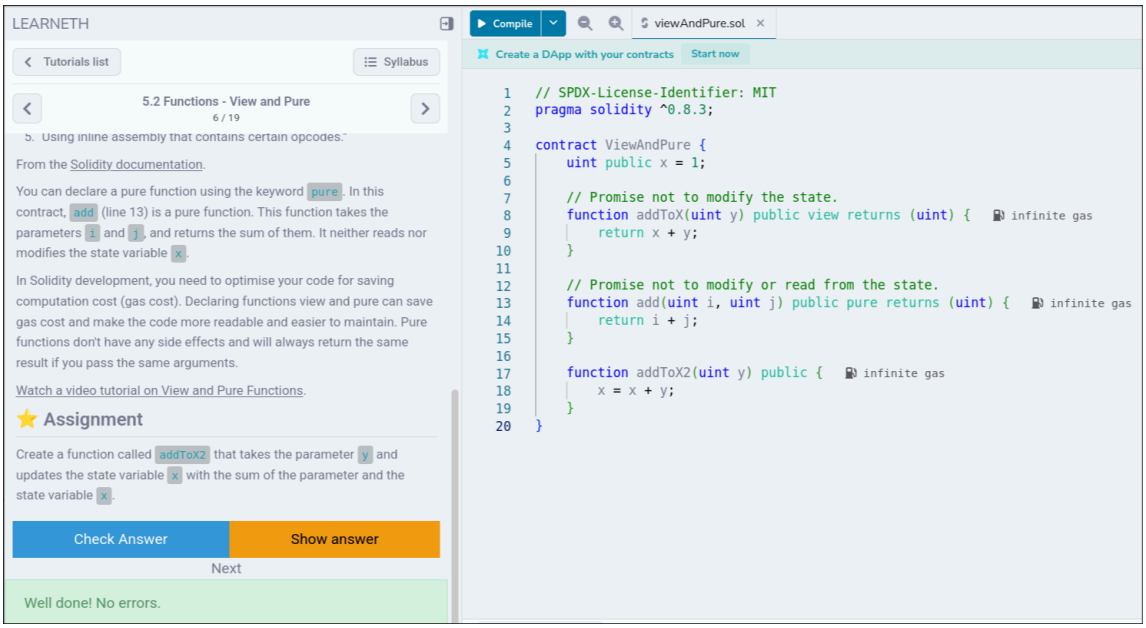
Step 5 — Functions: reading and writing a state variable

The SimpleStorage contract has set(uint) to write num and get() to read it. A public bool b = true and a function get_b() returning it are added.



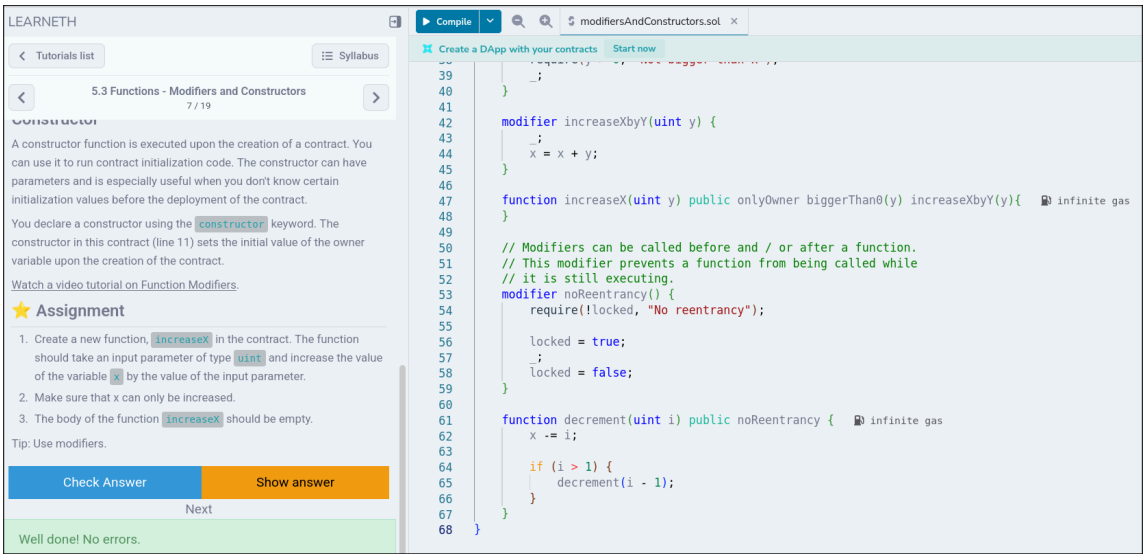
Step 6 — View and pure functions

addToX is a view function and add is a pure function. The assignment function addToX2(y) updates x with the sum of x and y.



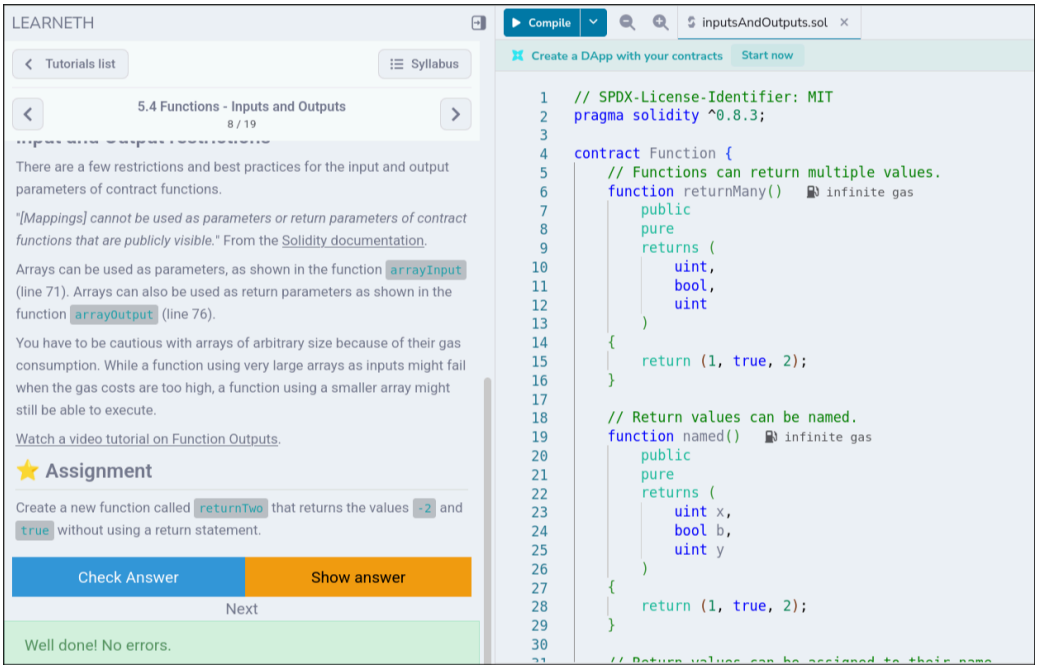
Step 7 — Modifiers and constructors

A constructor sets the initial owner and modifiers such as onlyOwner and noReentrancy are used. The assignment function increaseX is restricted by a modifier so that x can only be increased.



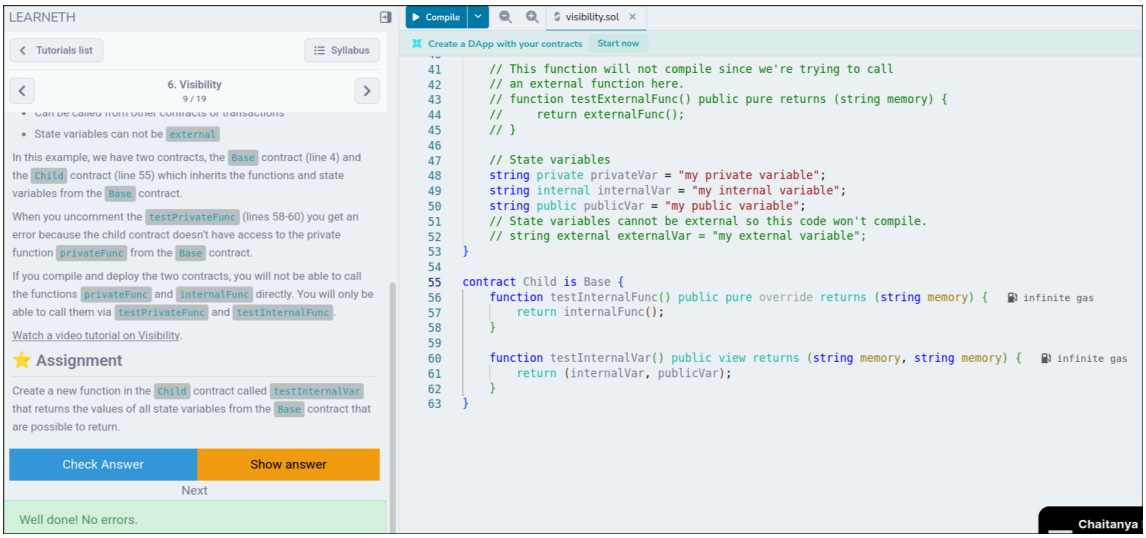
Step 8 — Function inputs and outputs

Functions return multiple and named values. The assignment function returnTwo returns -2 and true without using a return statement.



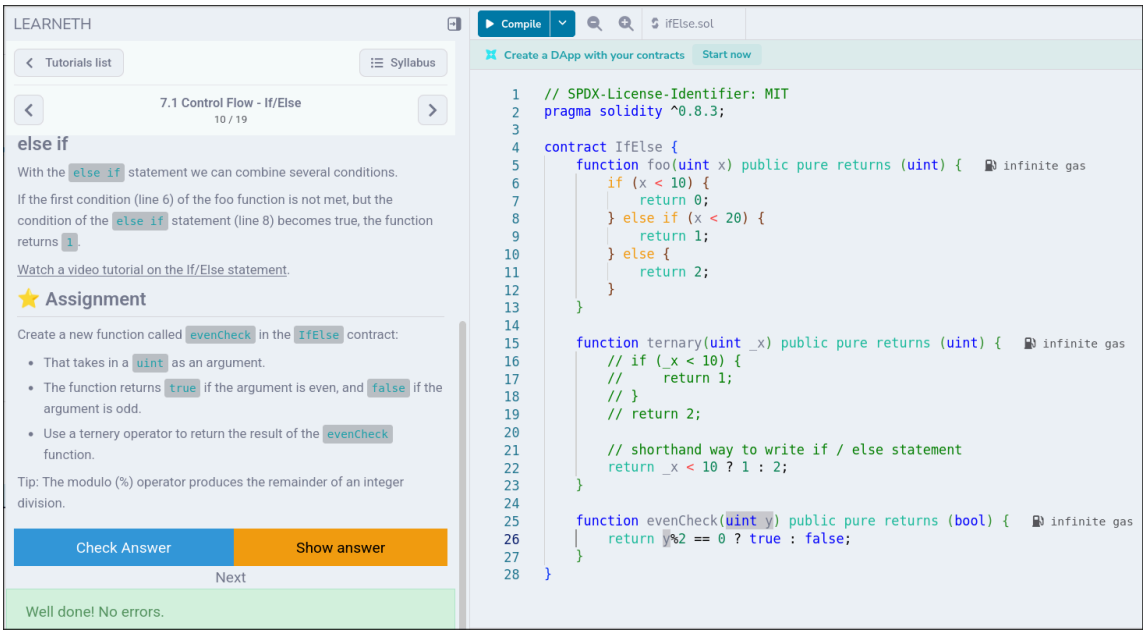
Step 9 — Visibility

The Base contract has private, internal and public variables and functions, and the Child contract inherits from it. testInternalVar returns all the state variables the child can access.



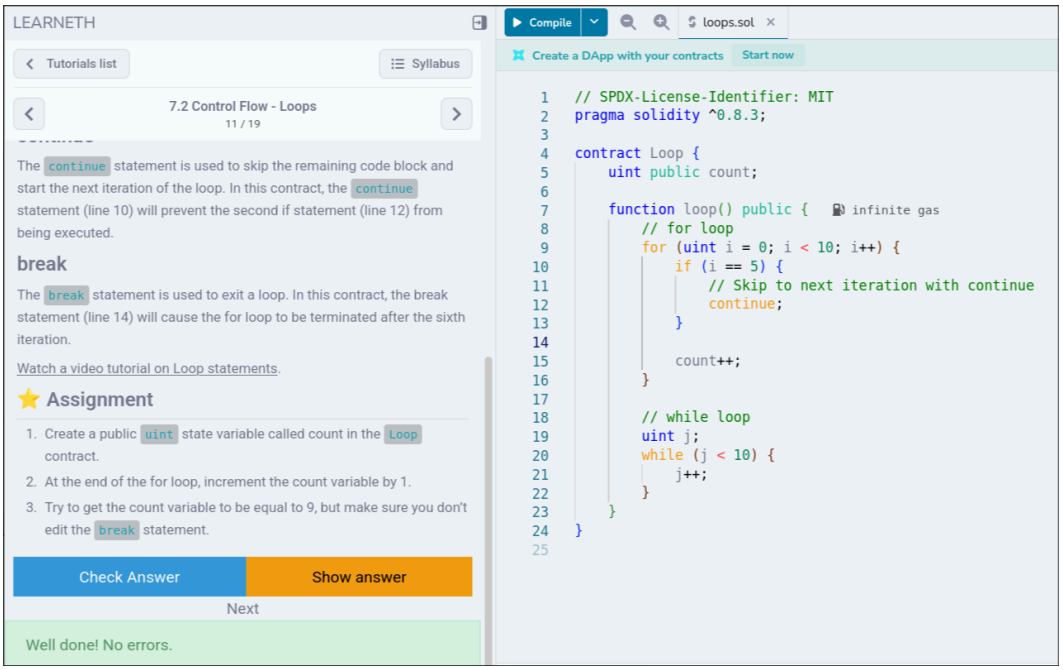
Step 10 — Control flow: if / else

foo uses if, else if and else, and ternary shows the shorthand form. The assignment function evenCheck returns true for an even number using the ternary operator.



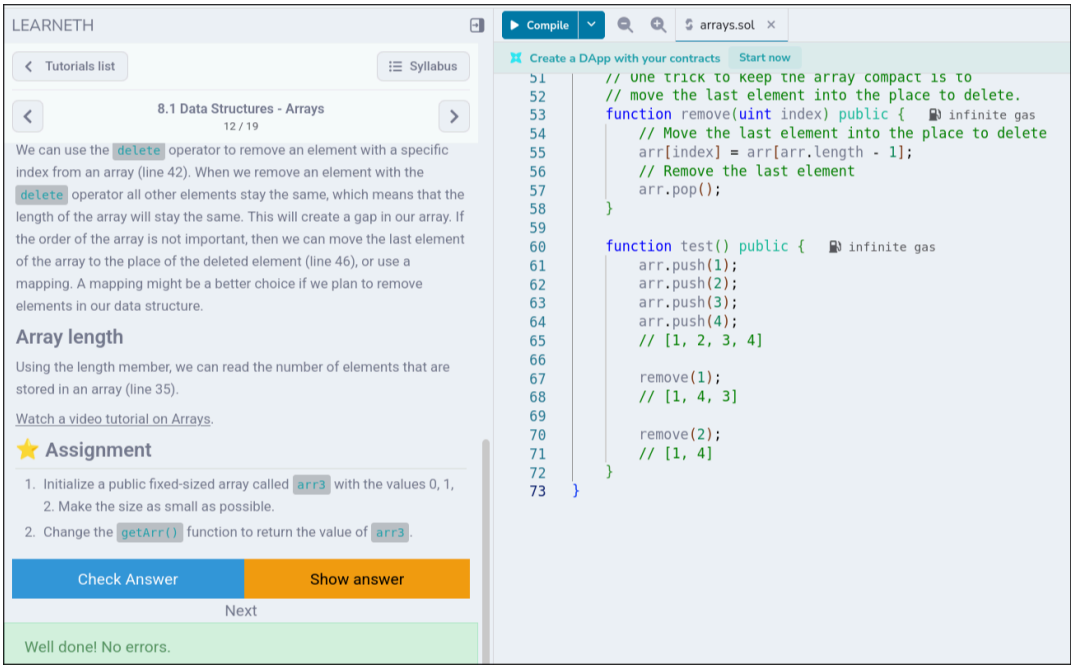
Step 11 — Control flow: loops

The Loop contract uses a for loop with continue and a while loop. The assignment increments the public count at the end of the for loop.



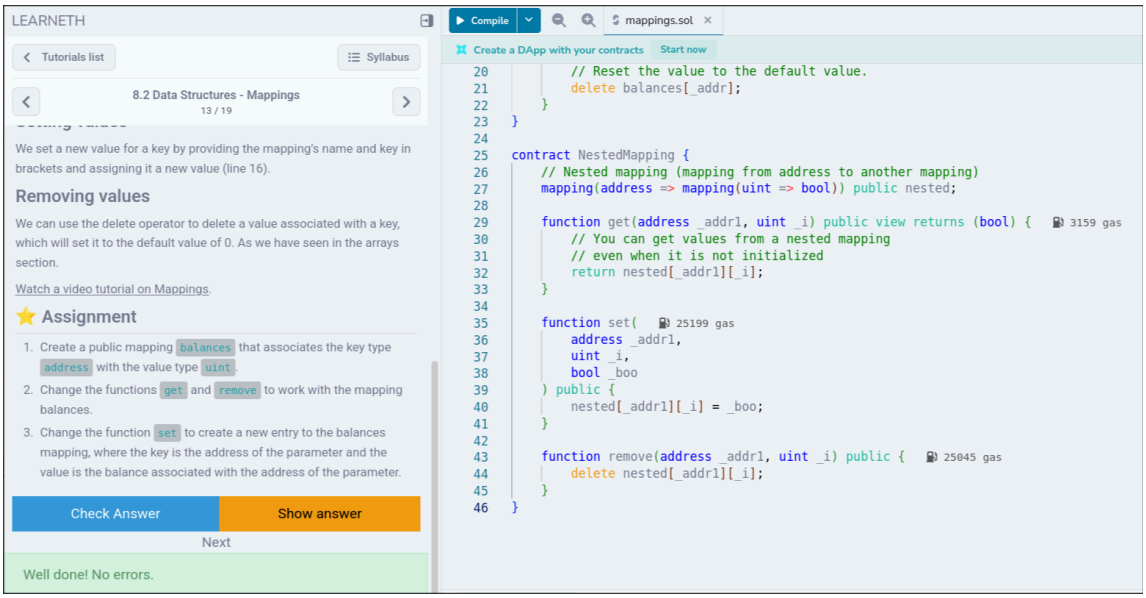
Step 12 — Data structures: arrays

An array is used with push, pop and delete, and remove(index) keeps the array compact by moving the last element into the deleted place. A fixed-size array arr3 is initialized with the values 0, 1, 2.



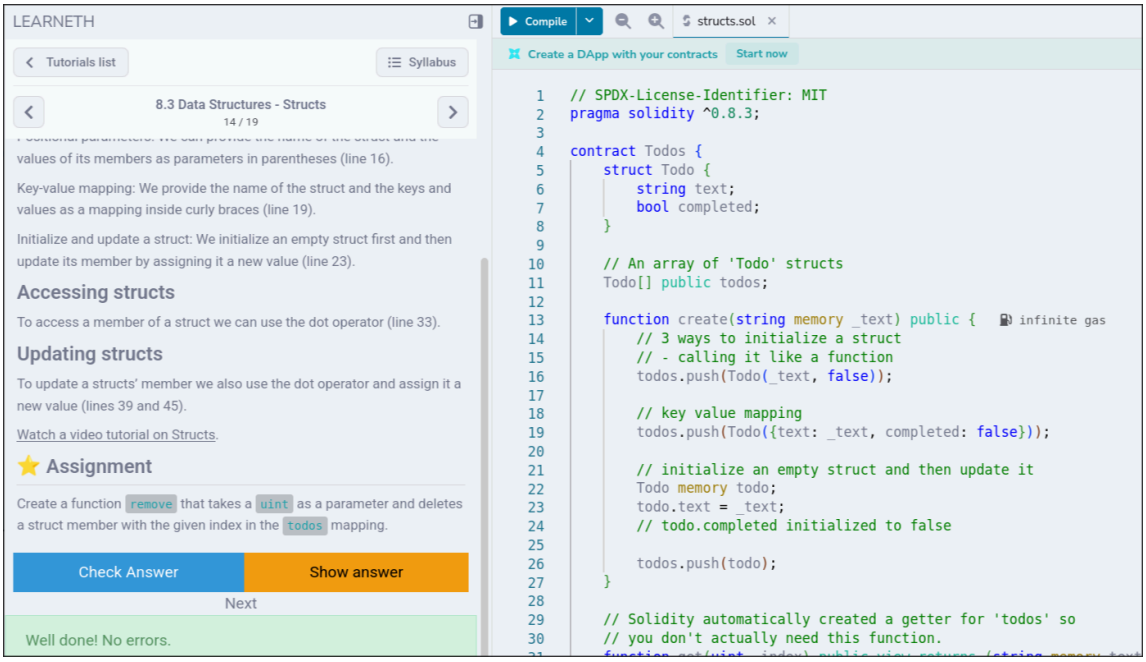
Step 13 — Data structures: mappings

A nested mapping from address to another mapping is created, with set, get and remove functions. The assignment creates a public mapping balances from address to uint.



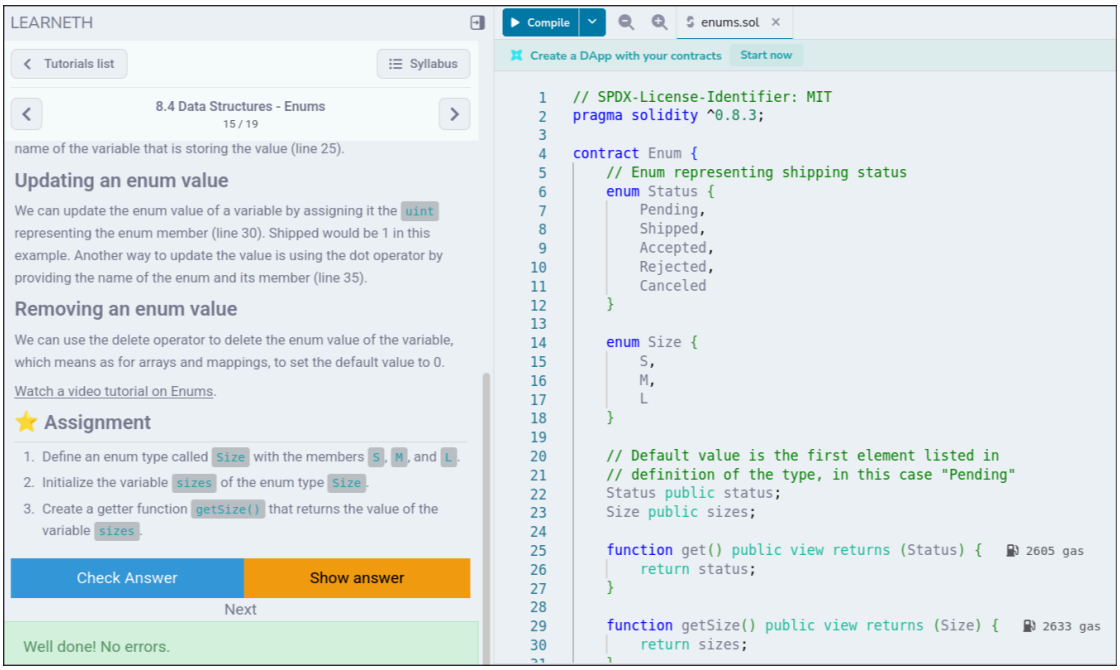
Step 14 — Data structures: structs

The Todos contract defines a Todo struct and shows three ways to create it. The assignment function remove deletes a struct from the todos mapping.



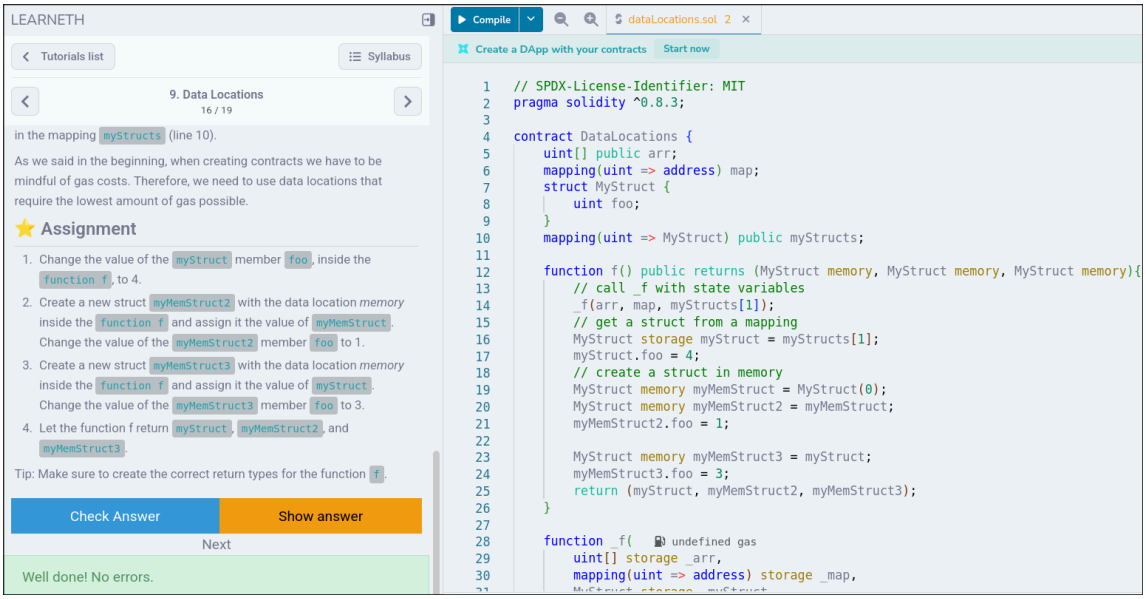
Step 15 — Data structures: enums

Enums Status and Size are defined with their members. The Size enum has S, M and L, and a getter getSize() returns the variable sizes.



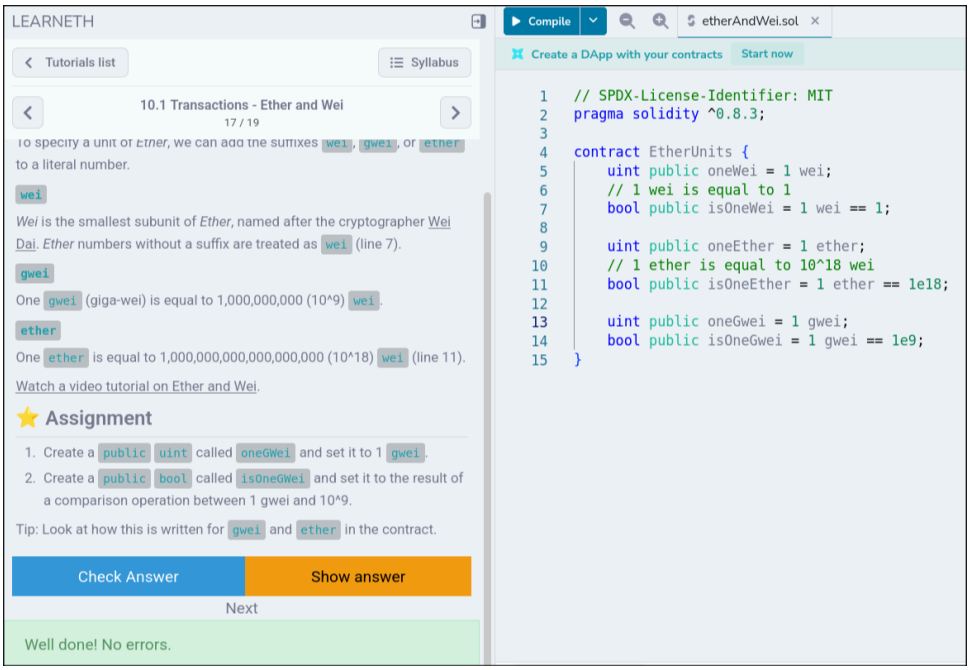
Step 16 — Data locations

The DataLocations contract shows storage and memory structs. The value of the foo member of myStruct is changed through storage and through memory copies.



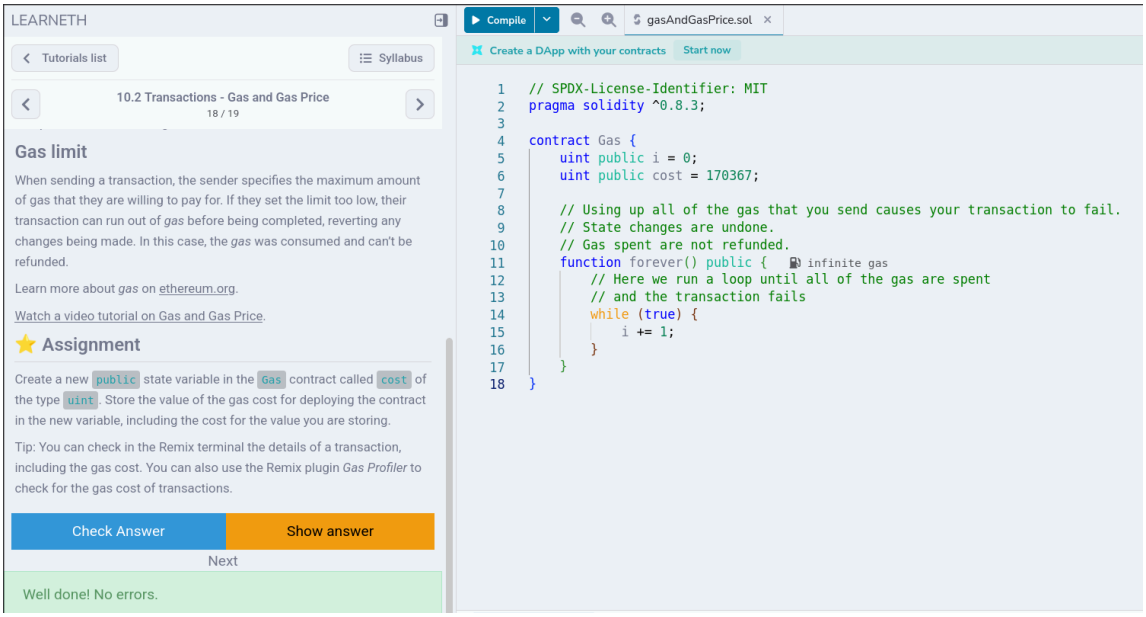
Step 17 — Transactions: Ether and wei

The EtherUnits contract shows 1 wei, 1 gwei and 1 ether. The assignment creates a public uint oneGwei = 1 gwei and a public bool isOneGwei comparing it with 1e9.



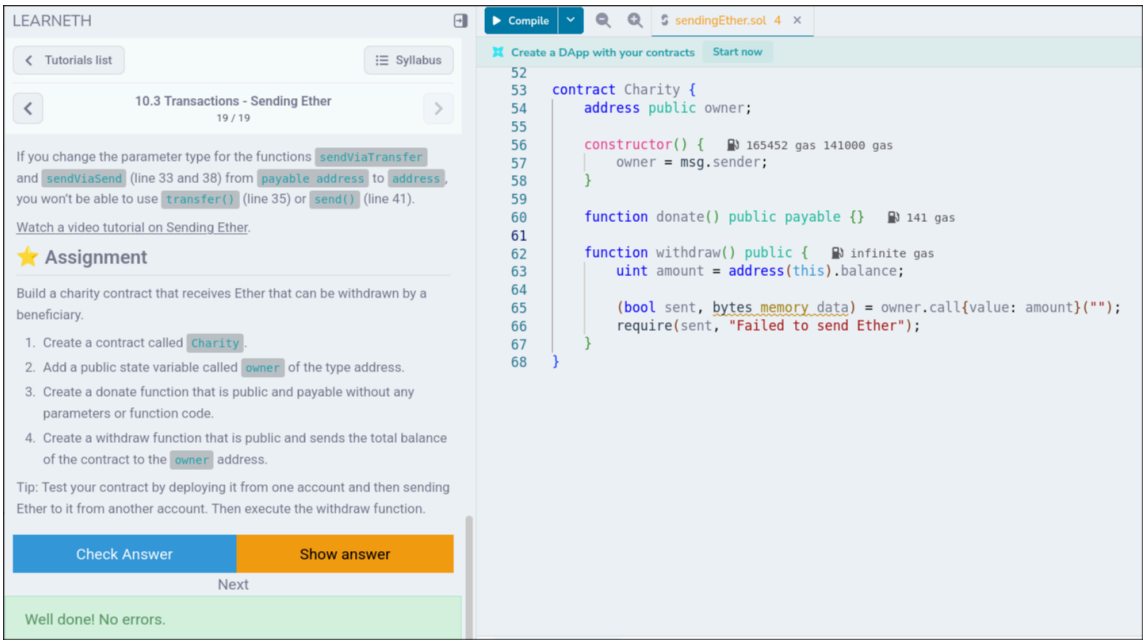
Step 18 — Transactions: gas and gas price

The Gas contract defines a public uint cost with the gas cost of deploying the contract, and a forever() function that loops until all gas is spent.



Step 19 — Transactions: sending Ether

A Charity contract is built with a public owner, a payable donate() function, and a withdraw() function that sends the total balance to the owner.



Conclusion:

The **Solidity Basics course** in Remix IDE was completed through hands-on assignments. The contracts covered **program structure, data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, data locations, and transaction concepts**, and each assignment was verified in Remix without errors. The experiment gave practical exposure to writing and testing smart contracts in Solidity.